Перколациони распоређивач

- Компактор

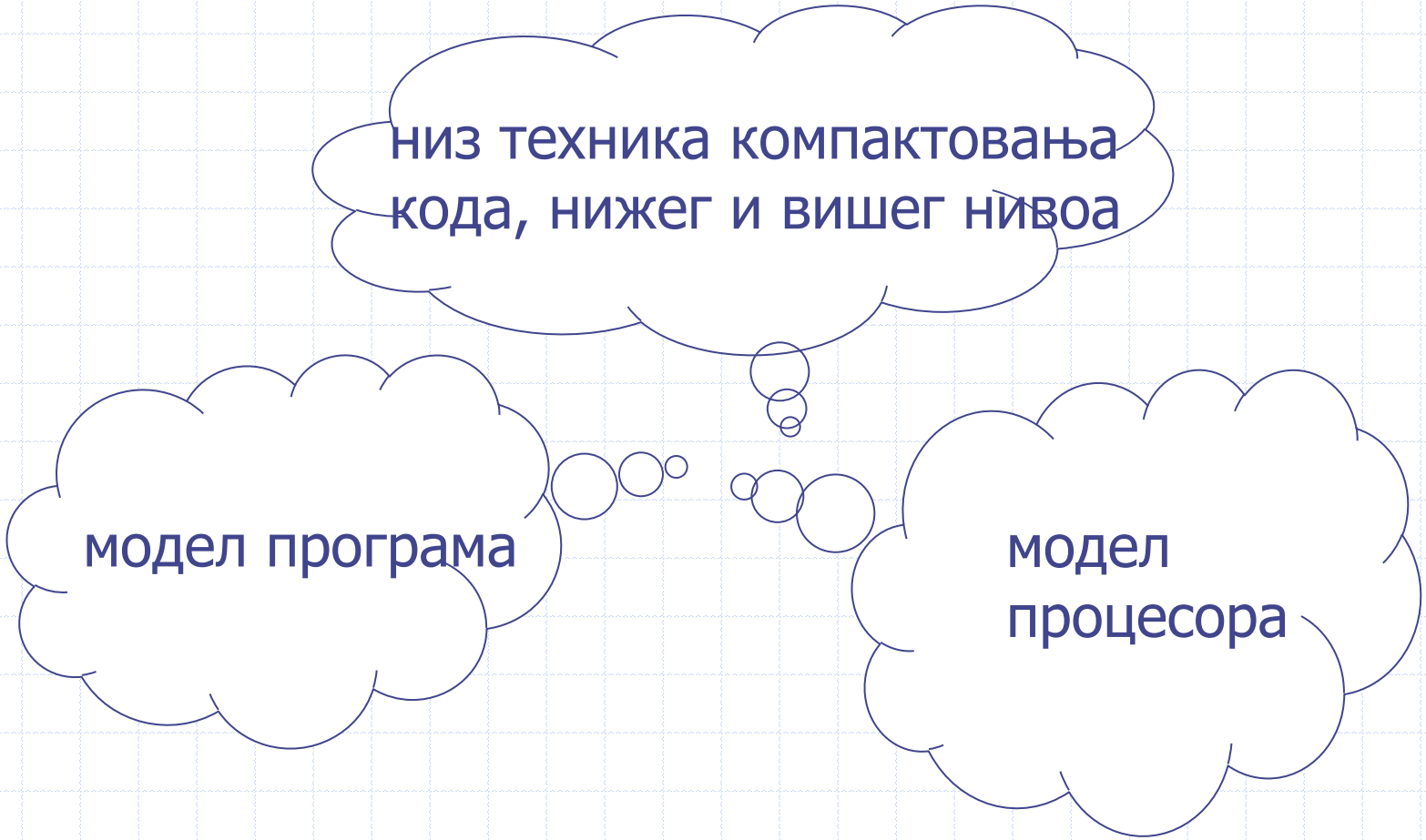
Компактор

- У скупу алата за MASC процесор, компактор је посебан алат.
- На тај начин, компактор се може применити и на ручно писаном коду.
- Међутим, то није суштински важно за разумевање улоге и начина рада компактора.

Функција Компактора

- Основни задатак компактора је да минимизира број машинских инструкција, тако што итеративно мења код и уклања непотребне машинске инструкције (пре свега NOP).
- Повољна последица ове минимизације је убрзање извршења програма, што иначе није примарни циљ.

Компоненте Компактора



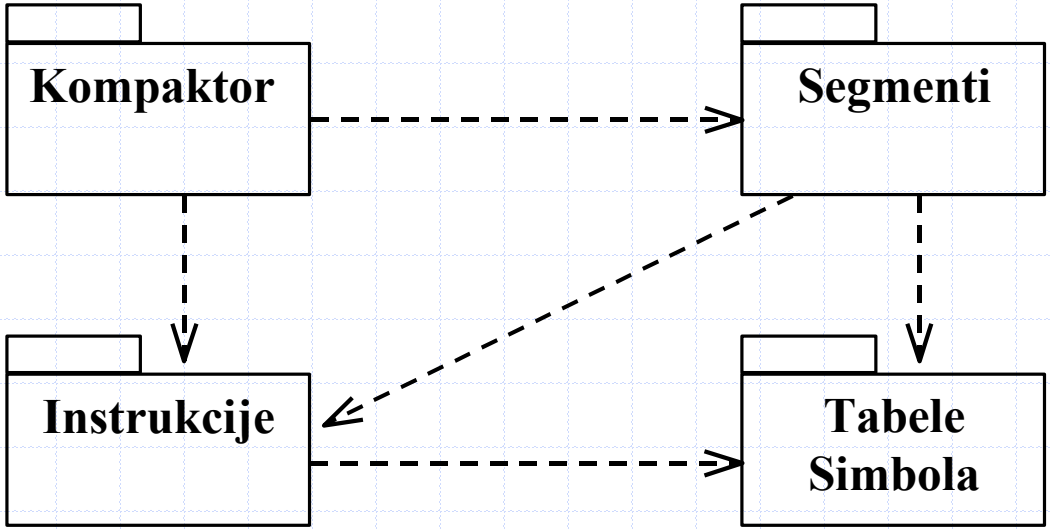
Основни захтев: очувати семантику програма!

- Основни захтев који компактор мора да задовољи је да у циљу оптимизације сме да мења програм, тј. његове трагове и трајекторију, али не сме да промени његово оригинално значење (његову функционалност).

Модел полазног програма

- Пошто је компактор посебан алат (да би се могао применити и на ручно писаном коду) онда на улазу добија машински код.
- Због тога, прво мора да реконструише инструкције у програмске објекте које онда организује у модел полазног програма. Тај модел личи на међурепрезентацију компајлера, али једноставнији и истиче само ствари које су битне за распоређивање.

Модел полазног програма

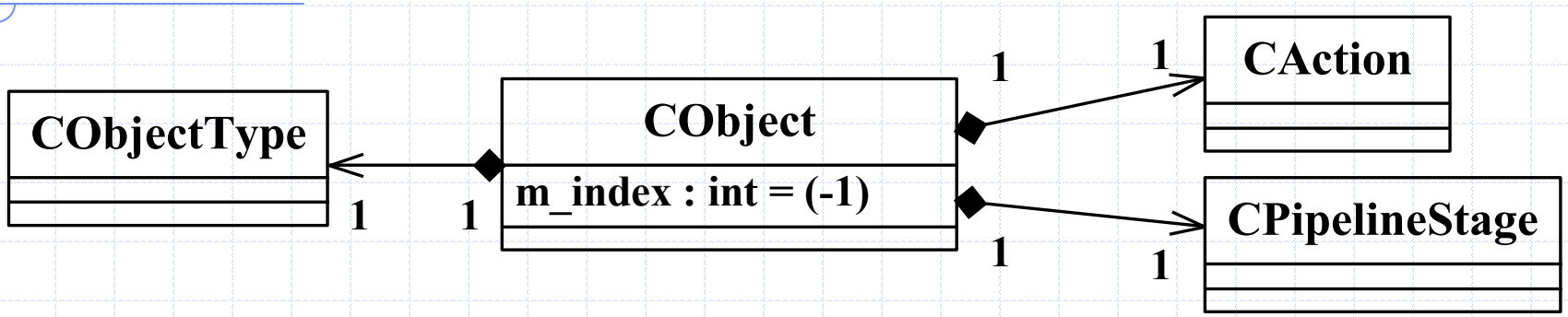


- Модел се састоји од скупа инстанци класа инструкција (изведених из основне класе CInstruction), који је сортиран по тзв. тачки извршења, неколико колекција лабела (програмске лабеле, лабеле података, симболичке константе и лабеле скокова), као и једне листе објеката програмских сегмената (инстанце класе CProgramSegment).

Уграђени модел процесора MASx

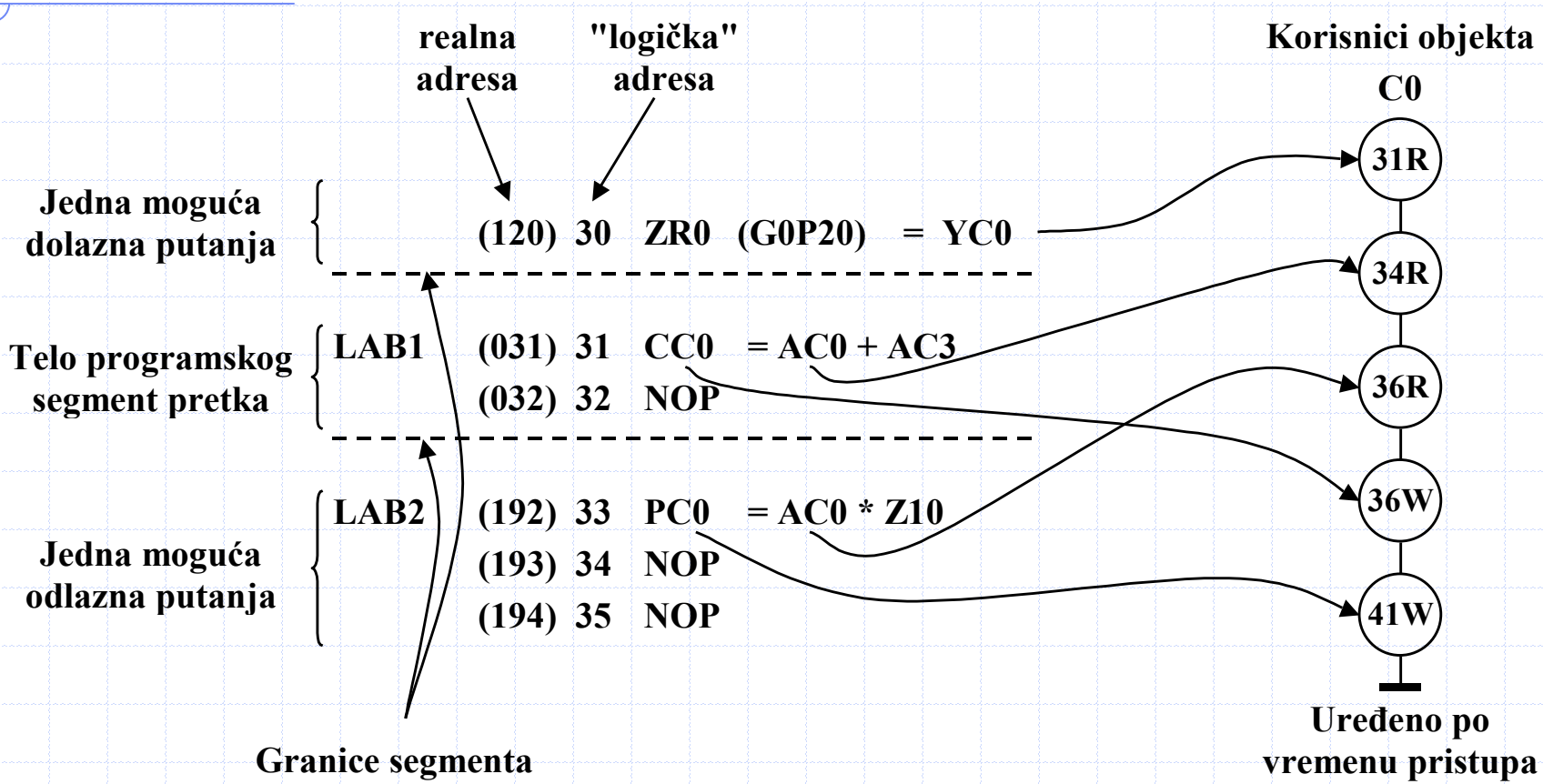
- Скуп класа изведених из *CInstruction*.
- За рад компактора кључна су два атрибута основне класе, и то:
- а) тачка извршења инструкције (*m_ExecutionPoint*), тј. њена адреса и
- б) низ објеката класе CObject, које инструкција користи на неки начин (m_Objects[]).

Модел ресурса – Класа *CObject*



- Кључни атрибути објекта су:
  - фаза проточне обраде (m_PipelineStage) у којој се објекат користи,
  - начин коришћења објекта: читање или упис (m_Action), и
  - тип објекта: акумулатор, регистар, итд. (m_ObjectType).

Адресни и простор објекти-време



Технике компактовања кода

- Компактор барата скупом техника које су хијерархијски организоване у два нивоа:
  - технике нижег нивоа раде на микро нивоу, тј. над једном инструкцијом или над паром инструкција
  - технике вишег нивоа које раде на марко нивоу, тј. над сегментима односно групама инструкција.

Технике нижег нивоа

- ЗАМЕНИ редослед задате инструкције са задатим NOP, на постојећим адресама, унутар сегмента,
- ИЗБАЦИ задати NOP,
- ЗАМЕНИ-ПРЕДХОДНИ: замени редослед задате инструкције сегмента са задатим NOP из претходног сегмента,
- ЗАМЕНИ-НАРЕДНИ: замени редослед задате инструкције сегмента са задатим NOP из наредног сегмента, и
- ПРЕИМЕНУЈ ресурс (акумулатор) са коришћеног имена на слободно,
- МУТИРАЈ инструкцију.

Технике вишег нивоа

- ТРАЖИ-УНАПРЕД,
- ТРАЖИ-УНАЗАД,
- КОМПАКТУЈ-СА-ПРЕДХОДНИМ, и
- КОМПАКТУЈ-СА-НАРЕДНИМ.

Технике нижег нивоа

- ЗАМЕНИ (инструкцију и NOP)
- ИЗБАЦИ (NOP)

- ЗАМЕНИ-ПРЕДХОДНИ
- ЗАМЕНИ-НАРЕДНИ

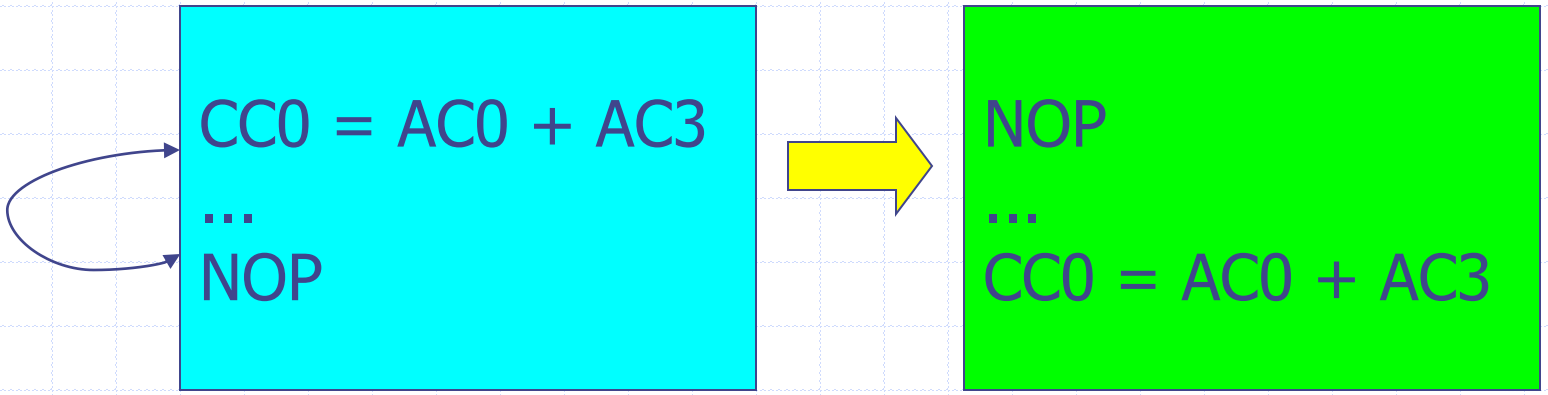
- ПРЕИМЕНУЈ
- МУТИРАЈ

Технике вишег нивоа

ТРАЖИ-УНАПРЕД
ТРАЖИ-УНАЗАД

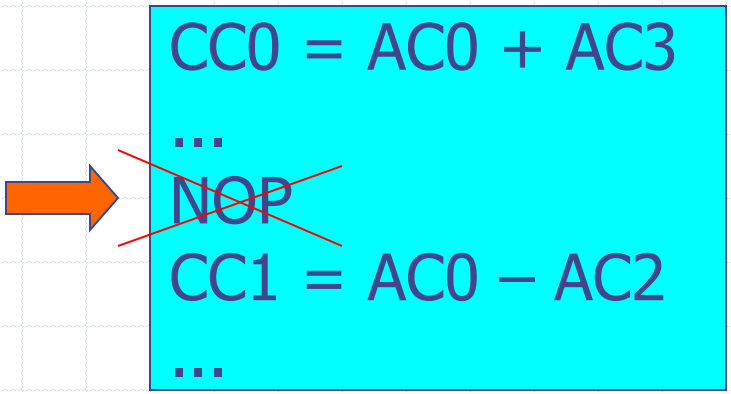
КОМПАКТУЈ-СА-ПРЕТХОДНИМ
КОМПАКТУЈ-СА-НАРЕДНИМ

ЗАМЕНИ инструкцију и NOP



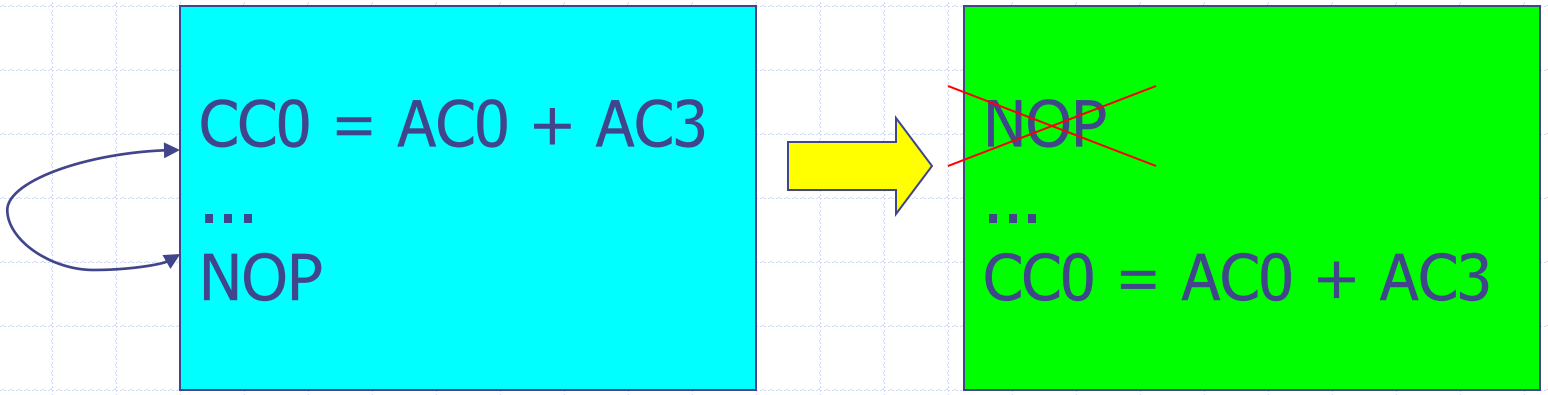
- Ова техника покушава да замени редослед задате инструкције и задатог NOP-а у истом сегменту (основном блоку), а да не промени значење програма (при том се консултује модел програма у простору објекти-време).

ИЗБАЦИ задати NOP



- Она покушава да ИЗБАЦИ задати NOP из програма, а да не промени значење програма. Уклањање NOP инструкције из низа инструкција, доводи до приближавања инструкција које следе NOP, инструкцијама које га претходе.

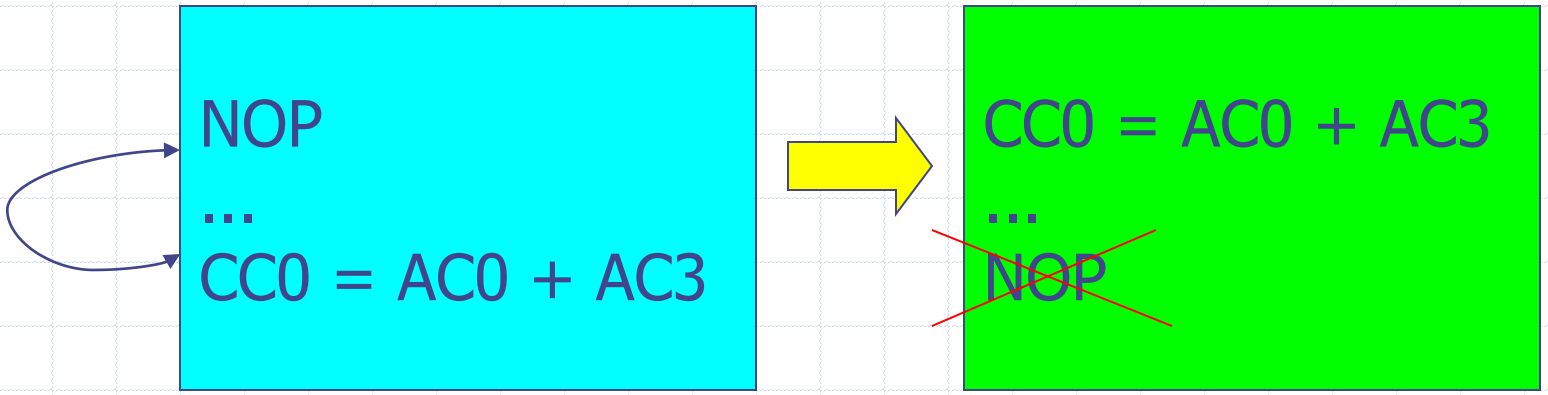
ТРАЖИ-УНАПРЕД



ЗАМЕНИ **ИЗБАЦИ**

- Ова техника вишег нивоа користи примитиве ЗАМЕНИ и ИЗБАЦИ. Полази од почетка програмског сегмента и у смеру његовог извршења (унапред) тражи NOP инструкцију.

ТРАЖИ-УНАЗАД



ЗАМЕНИ **ИЗБАЦИ**

- Полази од краја програмског сегмента и у смеру према почетку (уназад) тражи NOP инструкцију.

Модел тока управљања

- Ток управљања се моделује помоћу следеће три класе:
  - *Основни сегмент, CBasicProgramSegment*
  - *Сегмент, CProgramSegment*
  - *Виртуелни сегмент, CVirtualProgramSegment*

Основни програмски сегмент

- Моделује један изолован програмски сегмент, који је суштински једнак основном блоку.
- Његови атрибути, са становишта тока управљања, су почетна адреса сегмента *mStartAddress* и завршна адреса сегмента *mEndAddress*.

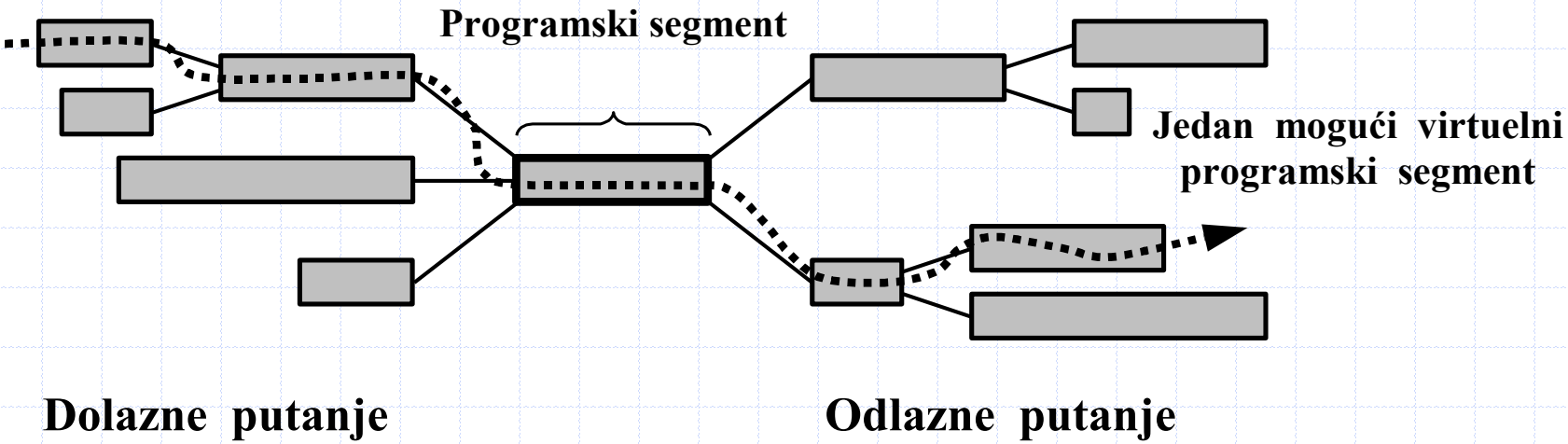
Програмски сегмент

- Моделује програмски сегмент као саставни део програма.
- Основна класа се проширује са три атрибута:
  - листа претходних програмских сегмената *mPreviousSegments*,
  - листа наредних сегмената *mNextSegments*, и
  - листа виртуелних програмских сегмената *mVirtualProgramSegments*.

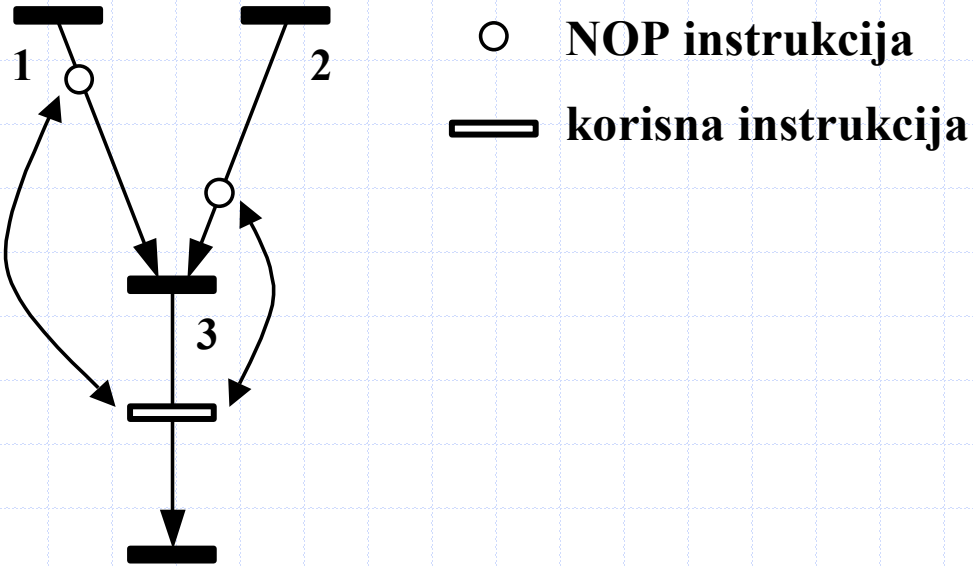
Виртуелни програмски сегмент

- *Виртуелни програмски сегмент* је нелинеарни сегмент (садржи и инструкције скока) који обухвата посматрани линеарни програмски сегмент.
- Виртуелни сегмент је део *трага*, који чине сегмент предак и онолико асемблерских инструкција пре и после сегмента предак колико фаза постоји у проточној обради (то се назива „дубина” проточне обраде).

Сегмент и Виртуални сегмент

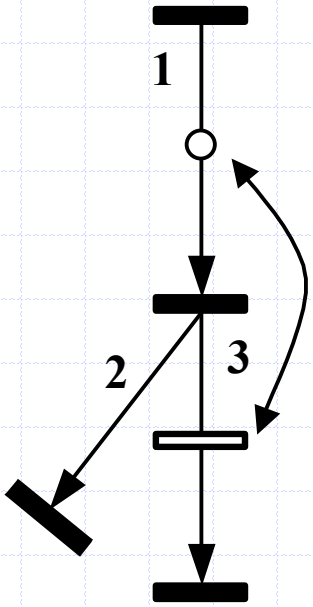


ЗАМЕНИ-ПРЕТХОДНИ



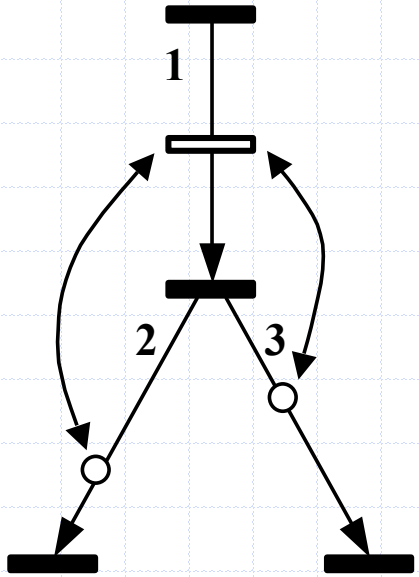
- Слична техници ЗАМЕНИ, уз две битне разлике: (а) контекст је: сегмент-преци и (б) у сваком претку се замењује по један NOP.

Пажња: без промене семантике!



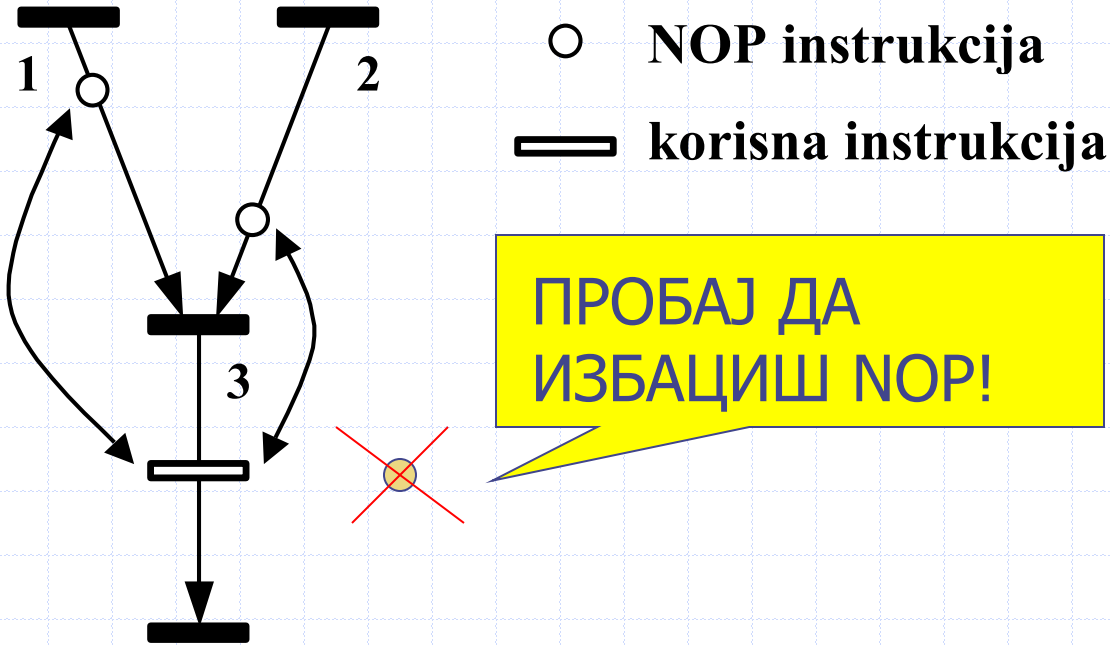
- Илустрација утицаја корисне инструкције на сегмент који је алтернативан сегменту предак.

ЗАМЕНИ-НАРЕДНИ



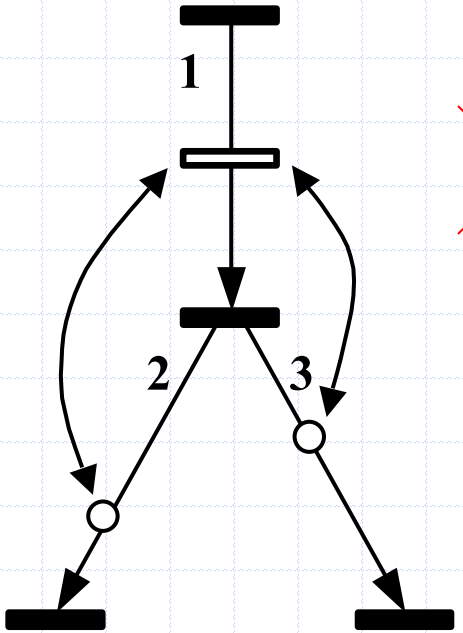
- Илустрација операције ЗАМЕНИ-НАРЕДНИ: корисна инструкција из сегмента 1 замењује NOP инструкције из сегмената 2 и 3.

КОМПАКТУЈ-СА-ПРЕТХНОДНИМ



- Ова техника вишег нивоа користи примитиве ЗАМЕНИ-ПРЕДХОДНИ и ИЗБАЦИ.

КОМПАКТУЈ-СА-НАРЕДНИМ



ПРОБАЈ ДА ИЗБАЦИШ NOP!

- Ова техника вишег нивоа користи примитиве ЗАМЕНИ-НАРЕДНИ и ИЗБАЦИ.

Учешљавање инструкција

задатак A:A1, NOP, A2, NOP ... , NOP, An

задатак B:B1, NOP, B2, NOP... , NOP, Bn

задатак A+B: A1, NOP, A2, NOP ... , NOP, An, B1, NOP, B2, NOP... , NOP, Bn

задатак A||B: A1, B1, A2, B2, ... , An, Bn

Задатак Компактора

- Пред компактор се, у ствари, поставља општи задатак да било који низ инструкција:

... Ii ... NOP ... Ij ... NOP ... Ik ... NOP

компактује тако што ће пронаћи минималан низ инструкција еквивалентног значења.

- Проблем *компактовања* кода је шири од проблема *учешљавања*, пошто је дозвољено коришћење било којих инструкција.

МУТИРАЈ инструкцију

- Могуће је да неки други облик инструкције, истог (или довољно сличног) значења може бити повољнији са аспекта бољег компактовања.
- Ако такве измене инструкција настану у фази компактовања, то се зове мутирање.

ПРЕИМЕНУЈ ресурс

- Замени ресурс у инструкцији слободним ресурсом истог типа.
- Показује се да ова техника, кад се комбинује са ЗАМЕНИ и ИЗБАЦИ, ствара прилике за додатно компактовање кода.

ПРЕИМЕНУЈ ресурс - пример

Са преименовањем

```
r1      = r2 + 1
[sp+8]  = r1
r1      = r5 + 2
[sp+12] = r1
```

→

```
r1      = r2 + 1
[sp+8]  = r1
r4      = r5 + 2
[sp+12] = r4
```

Без преименовања

```
r1      = r2 + 1
nop
[sp+8]  = r1
r1      = r5 + 2
nop
[sp+12] = r1
```

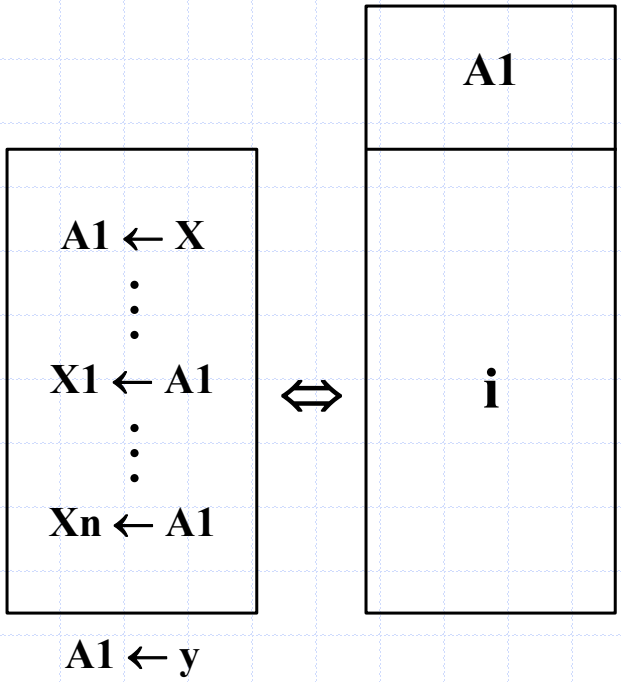
За проточну структуру

```
r1      = r2 + 1
r4      = r5 + 2
[sp+8]  = r1
[sp+12] = r4
```

За широку инст. реч

```
r1 = r2 + 1; r4 = r3 + 2
[sp+8]  = r1; [sp+12] = r4
```

Блок инструкција у ком се користи исти ресурс

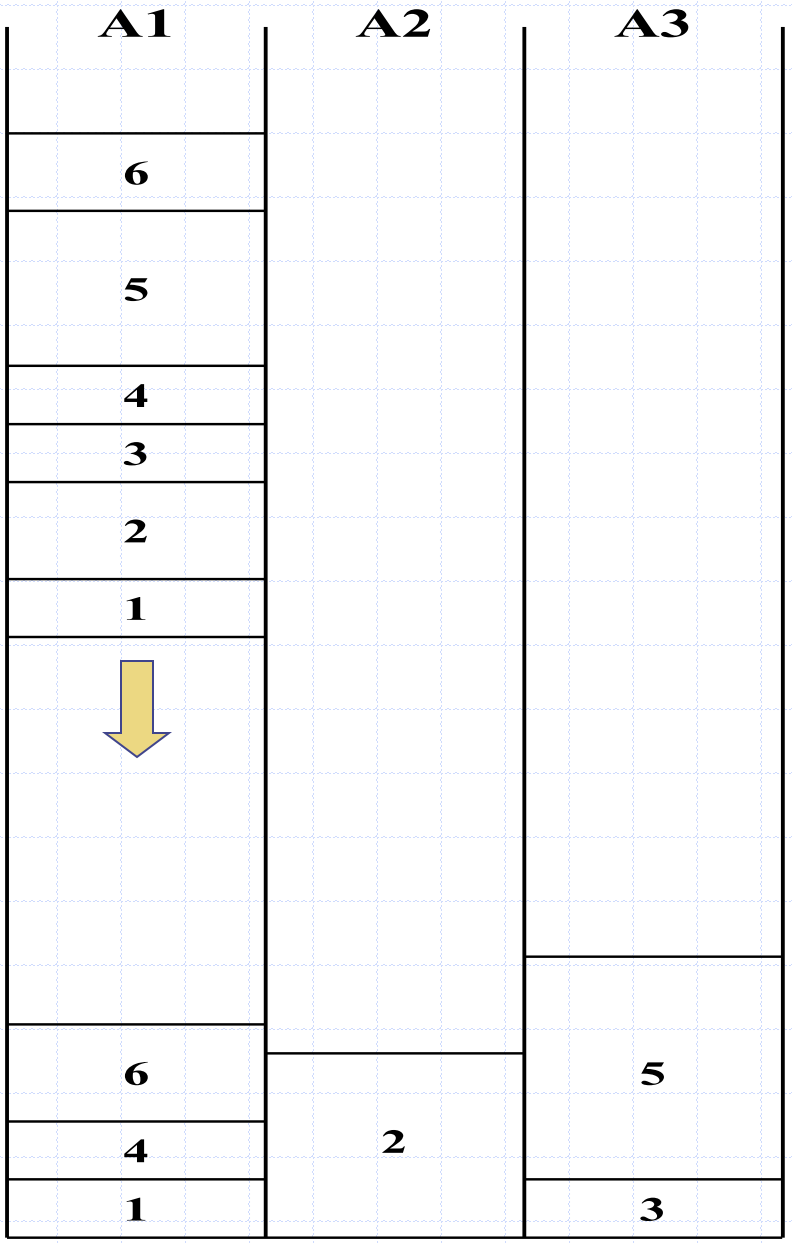


- Започиње инструкцијом пуњења акумулатора A1, а завршава се инструкцијом која претходи наредној инструкцији пуњења акумулатора A1

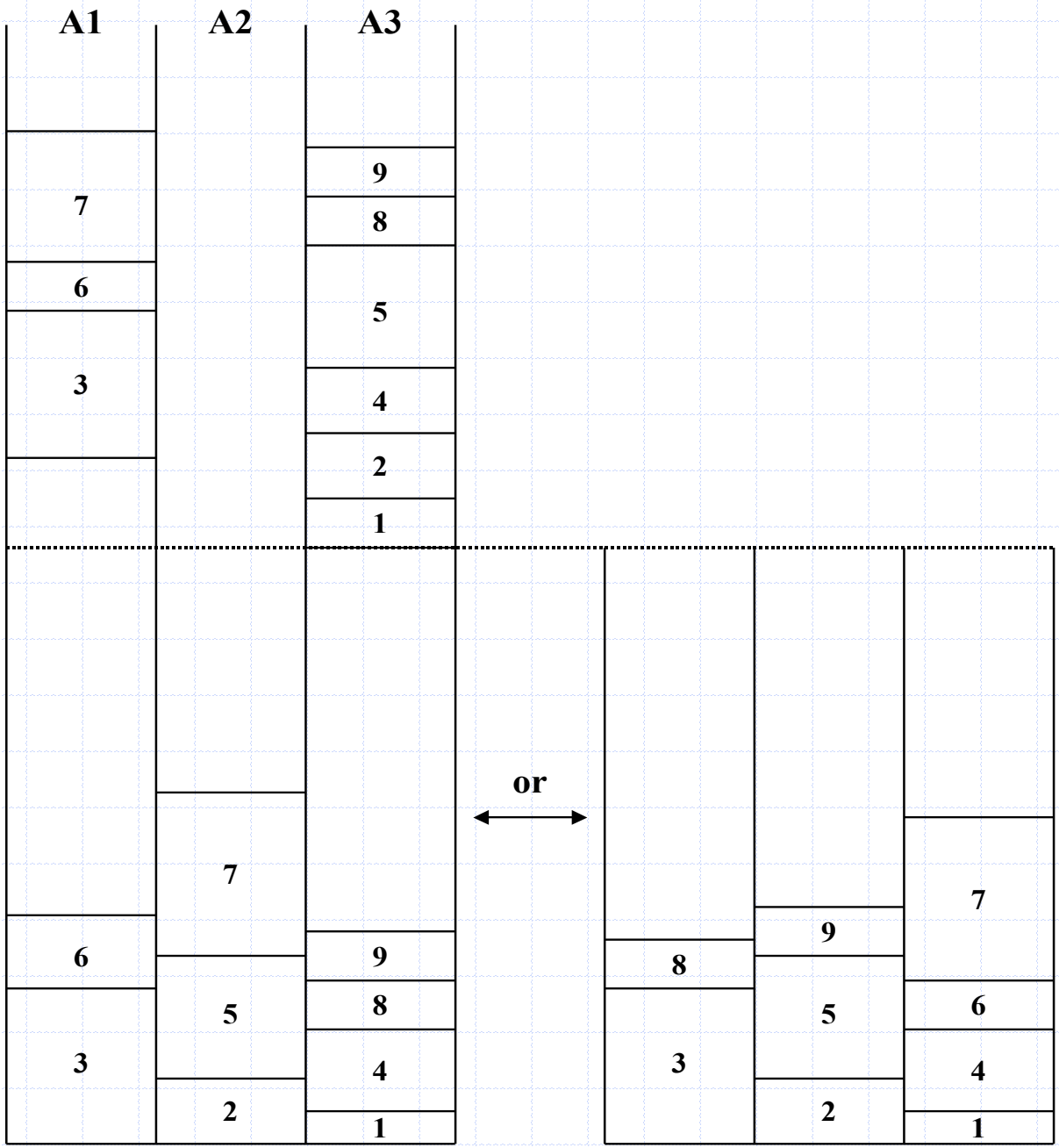
Преименовање, као Тетрис

БЛОКОВИ КОДА СЕ РАСПОРЕЂУЈУ НА ПРВИ СЛОБОДАН РЕСУРС!

КОРИСТИ СЕ „ПОХЛЕПАН” АЛГОРИТАМ.



Шта ако се, на пример, већ користе A1 и A3?



У пракси једна инструкција користи више ресурса, рецимо A1 и A3.

